

MoMo Lens: REST API Security, Documentation & DSA Report

Enterprise Web Development · Team Web Artisans (Kenny Louange, Maunice Akaliza, Jean Remy)

Component	Specification	Key Details
Course & Team	Enterprise Web Dev (ALU)	Team Web Artisans (@kennyrwigamba, @Akaliza-ux, @IREMY250)
Stack & Protocol	Pure Python 3.12 (http.server)	ThreadingHTTPServer; zero 3rd-party web dependencies
Data & Persistence	modified_sms_v2.xml (1,683 records)	In-memory hash index + atomic thread-safe JSON file (data/transactions.json)
Auth & Security	HTTP Basic Authentication	RFC 7617 credentials check; 401 challenge on failure; strict schema validation

1. Introduction to API Security

Financial Context: Mobile Money (MoMo) systems are critical financial conduits processing peer-to-peer transfers, merchant payments, cash-ins, and bill settlements. Unlike social or content platforms, a financial API exchanges actual currency value and highly sensitive Personally Identifiable Information (PII)—including account holder names, phone numbers, and real-time wallet balances. Securing this interface is paramount to prevent fraud, financial loss, and regulatory non-compliance.

Core Security Principles: Modern API security relies on five core pillars applied to the MoMo Lens gateway:

- **Confidentiality (Privacy):** Financial payloads and user balances are shielded from eavesdropping via transport encryption (TLS 1.3).
- **Integrity (Tamper Resistance):** Ensuring amounts, phone numbers, and ledger records cannot be modified or forged in transit or storage.
- **Availability (Resilience):** Protecting endpoints from denial-of-service and algorithmic resource exhaustion under heavy traffic.
- **Authentication & Access Control:** Verifying caller identity on every request and enforcing strict boundaries to prevent Broken Object-Level Authorization (BOLA/IDOR).
- **Non-Repudiation & Auditability:** Recording immutable, timestamped traces for every transaction created, updated, or deleted.

Threats & Mitigations: Primary threat vectors for financial APIs include BOLA/IDOR (modifying another user's transaction ID), packet sniffing over unencrypted channels, replay attacks (re-submitting identical payments), and malformed payload injection. MoMo Lens mitigates these through strict input schema validation (api/schemas.py), path-to-body ID verification on updates, duplicate ID conflict rejection (HTTP 409), and thread-safe atomic data writes (api/db.py).

2. Documentation of Endpoints

API Overview: The MoMo Lens API runs on `http://127.0.0.1:8000` (configurable via `MOMO_LENS_PORT`). All write requests require `Content-Type: application/json`. All routes require HTTP Basic Authentication (Authorization: Basic <base64(user:pass)>).

Method	Endpoint Path	Success	Error Codes	Action & Description
GET	/transactions	200 OK	401	List all transactions (supports

				?limit=&offset=)
GET	/transactions/{id}	200 OK	401, 404	Retrieve a specific transaction by unique ID
POST	/transactions	201 Created	400, 401, 409	Create/ingest a new transaction (validates full schema)
PUT	/transactions/{id}	200 OK	400, 401, 404	Update transaction (path ID must match body ID)
DELETE	/transactions/{id}	204 No Content	401, 404	Delete transaction from active ledger

2.1 Transaction JSON Schema

Schema Validation: All transactions adhere to the domain model in `api/schemas.py`: `transaction_id` (string 1-64 chars), `sender` & `receiver` (objects with `user_id`, `phone_number`, `full_name`, `user_type`), `amount` (> 0), `currency` (3-letter code, e.g. 'RWF'), `fee` (≥ 0), `balance_after` (≥ 0), `tx_timestamp` (ISO 8601), `status` (COMPLETED|PENDING|FAILED|REVERSED), `categories` (array of category objects), and optional `raw_sms_body`.

```
{
  "transaction_id": "13947831685",
  "sender": {"user_id": 1, "phone_number": "250788110381", "full_name": "Account Owner", "user_type": "INDIVIDUAL"},
  "receiver": {"user_id": 5, "phone_number": "250795963036", "full_name": "PayPal PAYMENT LTD", "user_type": "MERCHANT"},
  "amount": 25000.0, "currency": "RWF", "fee": 250.0, "balance_after": 40930.0,
  "tx_timestamp": "2026-09-14T10:10:00Z", "status": "COMPLETED",
  "categories": [{"category_id": 2, "category_code": "MERCHANT_PAYMENT", "category_name": "Merchant Payment", "is_primary": true}]
}
```

2.2 Detailed Endpoint Specifications

GET /transactions: Returns an array of all transactions in the ledger. Header: Authorization: Basic <base64>. Response: 200 OK (JSON array). Error: 401 Unauthorized if auth is missing/invalid.

```
curl -X GET http://127.0.0.1:8000/transactions -H 'Authorization: Basic YWRtaW46c2VjcmV0MTIz'
```

GET /transactions/{id}: Fetches a single transaction matching {id}. Response: 200 OK with transaction object. Errors: 401 Unauthorized; 404 Not Found ({"error": "Transaction not found"}).

```
curl -X GET http://127.0.0.1:8000/transactions/13947831685 -H 'Authorization: Basic YWRtaW46c2VjcmV0MTIz'
```

POST /transactions: Ingests a new transaction. Body: Full transaction JSON object. Response: 201 Created (echoes created object). Errors: 400 Bad Request (invalid JSON or validation failure, e.g. `amount <= 0`); 401 Unauthorized; 409 Conflict ({"error": "Transaction already exists"} if ID duplicate).

```
curl -X POST http://127.0.0.1:8000/transactions \
  -H 'Authorization: Basic YWRtaW46c2VjcmV0MTIz' -H 'Content-Type: application/json' \
  -d '{"transaction_id": "TX_9901", "sender": {"user_id": 1, "phone_number": "250788110381", "full_name": "Kenny L", "user_type": "INDIVIDUAL"}, "receiver": {"user_id": 2, "phone_number": "250788556677", "full_name": "Merchant LTD", "user_type": "MERCHANT"}, "amount": 15000.0, "currency": "RWF", "fee": 150.0, "balance_after": 85000.0, "tx_timestamp": "2026-09-29T14:30:00Z", "status": "COMPLETED", "categories": [{"category_id": 1, "category_code": "P2P", "category_name": "Transfer"}]}'
```

PUT /transactions/{id}: Updates an existing transaction. Consistency Check: Path {id} must strictly match payload transaction_id (prevents ID spoofing / IDOR). Response: 200 OK (updated JSON). Errors: 400 Bad Request (ID mismatch or invalid schema); 401 Unauthorized; 404 Not Found.

```
curl -X PUT http://127.0.0.1:8000/transactions/TX_9901 \
-H 'Authorization: Basic YWRtaW46c2VjcmV0MTIz' -H 'Content-Type: application/json' \
-d '{"transaction_id": "TX_9901", "sender": {"user_id": 1, "phone_number": "250788110381", "full_name": "Kenny L", "user_type": "INDIVIDUAL"}, "receiver": {"user_id": 2, "phone_number": "250788556677", "full_name": "Merchant LTD", "user_type": "MERCHANT"}, "amount": 18000.0, "currency": "RWF", "fee": 180.0, "balance_after": 82000.0, "tx_timestamp": "2026-09-29T14:30:00Z", "status": "COMPLETED", "categories": [{"category_id": 1, "category_code": "P2P", "category_name": "Transfer"}]}'
```

DELETE /transactions/{id}: Permanently removes a transaction from both the in-memory index and the persistent JSON file. Response: 204 No Content (empty body). Errors: 401 Unauthorized; 404 Not Found.

```
curl -X DELETE http://127.0.0.1:8000/transactions/TX_9901 -H 'Authorization: Basic YWRtaW46c2VjcmV0MTIz'
```

2.3 Error Code Reference

HTTP Code	Status Text	Typical Cause in MoMo Lens	Error Payload Format
400	Bad Request	Malformed JSON, missing fields, negative amount, or path ID mismatch	{"error": "<reason>"}
401	Unauthorized	Missing Authorization header or wrong credentials	{"error": "Unauthorized: Invalid or missing credentials"}
404	Not Found	Transaction ID does not exist, or unmapped URL path	{"error": "Transaction not found"}
409	Conflict	POST submitted with an existing transaction_id	{"error": "Transaction already exists"}
500	Internal Error	Unexpected I/O failure or uncaught server exception	{"error": "Internal server error occurred"}

3. Results of DSA Comparison

Objective: To optimize API latency for GET, PUT, and DELETE /transactions/{id}, we compared two data structures across the full dataset of 1,683 parsed MoMo transactions from modified_sms_v2.xml: (1) Linear Search over list[dict], and (2) Hash Table Lookup over dict[str, dict].

Algorithm	Data Structure	Best Case	Average Case	Worst Case	Auxiliary Space
Linear Search	Python list (list[dict])	O(1) (index 0)	O(n/2) ≈ O(n)	O(n) (end or missing)	O(1) extra space
Dictionary Lookup	Python hash map (dict[str, dict])	O(1)	O(1) amortized	O(n) (hash collision)	O(n) hash table

Benchmark Methodology: Using `dsa/benchmark.py`, we evaluated 24 test cases (exceeding the >=20 rubric requirement) across early, middle, late, and non-existent IDs. Each lookup was executed over 500 repeated cycles using `time.perf_counter_ns()`.

Position / Scenario	Target ID	Linear Search (µs)	Dict Lookup (µs)	Speedup
---------------------	-----------	--------------------	------------------	---------

				Factor
Start (Index 0)	76662021700	0.111 µs	0.065 µs	1.7×
Early (Index 100)	sms_107	3.722 µs	0.062 µs	60.4×
Middle (Index 850)	sms_859	40.262 µs	0.065 µs	615.6×
Late (Index 1500)	17878271668	72.649 µs	0.067 µs	1,090.8×
End (Index 1682)	37832903831	73.276 µs	0.070 µs	1,040.8×
Missing / 404 (Worst Case)	NOT_FOUND_90001	57.187 µs	0.064 µs	896.3×
Overall Average (24 runs)	Across all test cases	40.809 µs	0.066 µs	619.4× faster

Key Takeaway: Linear search degrades linearly from 0.11 µs at index 0 to 73.28 µs at index 1682. Crucially, querying non-existent IDs forces linear search to inspect all 1,683 records (taking ~57 µs), exposing the API to algorithmic DoS (404 flooding). Dictionary lookup delivers flat O(1) performance (~0.066 µs) across all positions and missing keys, achieving a 619.4× average speedup and ensuring predictable API latency.

4. Reflection on Basic Auth Limitations

Overview: In Sprint 3, HTTP Basic Authentication (RFC 7617) protects the API by passing Authorization: Basic <base64(user:pass)>. While simple to implement in standard Python, Basic Auth has critical vulnerabilities that make it unsuitable for production financial gateways:

- **Base64 is NOT Encryption:** Base64 is trivial to decode in zero CPU cycles without a key. If TLS terminates or traffic is captured, credentials are exposed in plaintext.
- **Credentials Sent on Every Request:** Statelessness requires transmitting master credentials with every single API call, multiplying exposure across network hops, reverse proxies, and log files.
- **No Expiration or Session Revocation:** Basic Auth lacks a time-to-live (TTL). If an agent or mobile device is compromised, an administrator cannot revoke that specific session without resetting the master password system-wide.
- **No Granular Scopes or RBAC:** Basic Auth is binary (authenticated or not). It cannot differentiate permissions between a customer (read-only balance), a cashier (create transactions), and an auditor (read audit logs).
- **Aggressive Browser Caching:** Browsers persistently cache Basic Auth credentials in memory until the application is fully closed, making secure programmatic client-side logout impossible.

4.1 Recommended Enterprise Alternatives

Alternative Mechanism	Core Mechanism	Key Advantage for MoMo Lens
JSON Web Tokens (JWT) [RFC 7519]	Cryptographically signed tokens (HMAC/RSA) with embedded expiration and role claims.	Stateless validation, 15-min token expiration, fine-grained role scopes (RBAC), prevents perpetual credential exposure.
OAuth 2.0 / OIDC [RFC 6749]	Decoupled authorization server issuing scoped Access Tokens & Refresh Tokens.	Standardized token revocation (RFC 7009), delegated third-party merchant permissions, and MFA integration.
mTLS + HMAC Request Signing	Bidirectional x509 certificates and HMAC-SHA256 signature over payload + timestamp.	Mandatory for inter-bank and telco settlement rails; guarantees hardware identity proof and prevents replay attacks.